МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Информационных технологий»

Лабораторная работа № 7
по дисциплине «Введение в информационные технологии»
по теме: «**КАЧЕСТВО И НАДЁЖНОСТЬ PYTHON-КОДА: СТИЛЬ, СТАТИЧЕСКИЙ АНАЛИЗ, ОТЛАДКА И ТЕСТИРОВАНИЕ**»

Выполнил студент группы:
БВТ2601
Гутаев Эльдар Назирович
Проверил:

Москва 2026

Оглавление

Введение……3

Выполнение 4

Вывод..... 28

Введение

Цель и задачи работы:

Целью данной лабораторной работы является: изучить основные принципы написания читаемого, поддерживаемого и надёжного Python-кода; познакомиться с рекомендациями PEP 8; научиться применять автоматическое форматирование и статический анализ кода с помощью Ruff, использовать средства отладки IDE, находить логические ошибки, создавать автоматизированные тесты с использованием assert и pytest, а также выполнять базовый рефакторинг программного кода.

Задачи: В ходе выполнения лабораторной работы необходимо последовательно проанализировать и улучшить небольшой проект с намеренно заложенными проблемами: выявить недостатки оформления и именования, выполнить автоматическое форматирование и статический анализ, найти и исправить логические ошибки с помощью отладчика, разработать набор тестов и убедиться, что после рефакторинга поведение программы осталось корректным.

Выполнение

Задание 1. Создан каталог code_quality_lab (в общей папке он назван lab_07_code_quality_lab) со структурой из методических указаний: main.py, calculations.py, requirements.txt и tests/test_calculations.py. Ввод в терминал VS Code мне недоступен, поэтому команды оформлены как задачи VS Code (.vscode/tasks.json) и запускались через Terminal → Run Task. В задачах в начало PATH добавлен каталог .venv\Scripts — это то же, что делает активация окружения, поэтому команды python, ruff и pytest берутся из .venv; для тестов в PYTHONPATH указан корень проекта, чтобы модуль calculations импортировался из tests. На рисунке 1: создано окружение .venv, установлены ruff 0.17.3 и pytest 9.2.0 с зависимостями. На рисунке 1.1 — requirements.txt, сформированный pip freeze. PowerShell 5.1 записал его в кодировке UTF-16 (видно в строке состояния), pip такой файл читает.

```
python -m venv .venv
python -m pip install ruff pytest
python -m pip freeze > requirements.txt
```

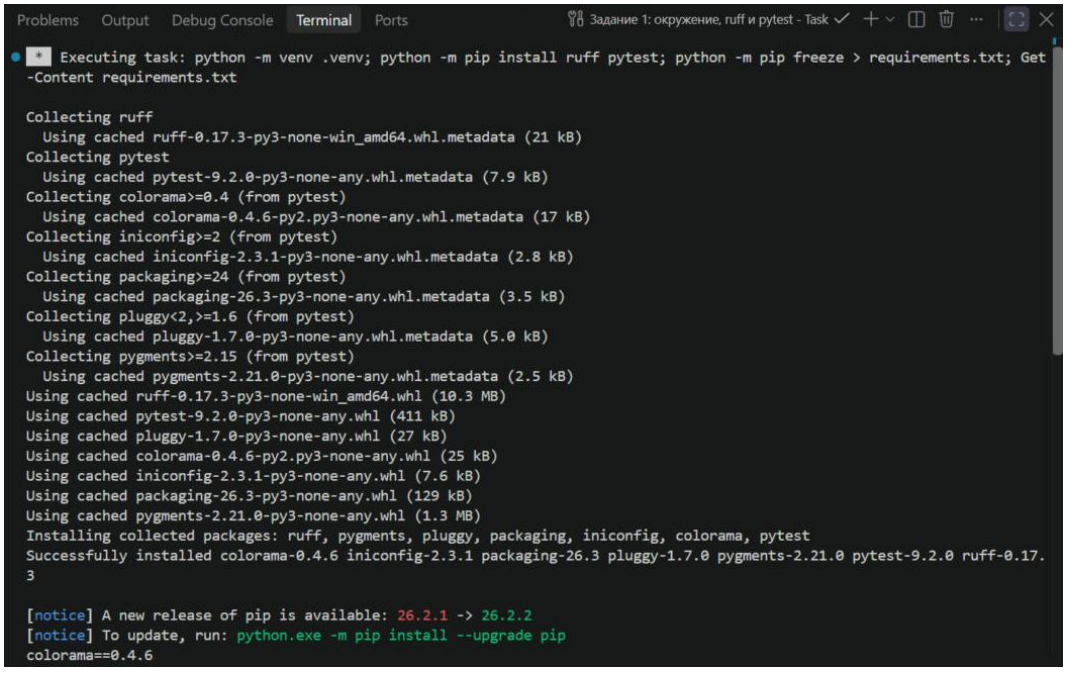


Рисунок 1. Создание окружения и установка Ruff и pytest

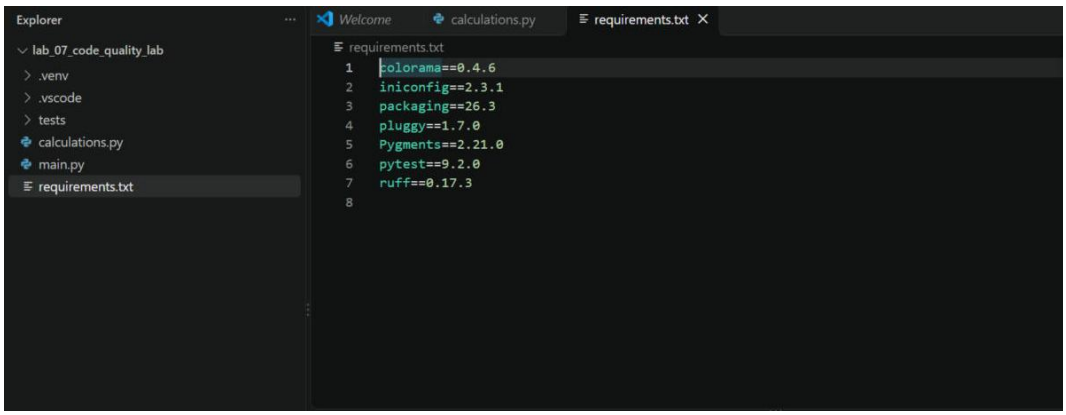


Рисунок 1.1. Структура проекта и requirements.txt

Задание 2. В calculations.py помещён исходный код без исправлений (рисунок 2). До запуска инструментов найдены следующие проблемы:

1) import math и import json не используются — удалить оба импорта;

2) в функции calc отступ 2 пробела вместо 4 (s=0, for x in a:) — привести к 4 пробелам по PEP 8;

3) нет пробелов вокруг операторов: s=s+x, x>m, c=c+1 — писать s = s + x, x > m, c = c + 1;

4) непонятные имена: calc, mx, countpos, a, s, m, c — заменить на calculate_average, find_max, count_positive, numbers, total, maximum, count;

5) логическая ошибка в calc: return s/(len(a)+1) делит на число элементов плюс один — нужно s / len(a);

6) логическая ошибка в mx: начальное m=0 больше любого отрицательного числа — начинать с первого элемента a[0];

7) countpos считает и ноль (x>=0), хотя ноль не положительное число — условие x > 0;

8) нет обработки пустого списка — после исправлений calc дал бы ZeroDivisionError, а mx — IndexError; нужно явно выбрасывать ValueError.

```
import math
import json


def calc(a):
  s=0
  for x in a:
      s=s+x
  return s/(len(a)+1)


def mx(a):
    m=0
    for x in a:
        if x>m:
            m=x
    return m


def countpos(a):
    c=0
    for x in a:
        if x>=0:
            c=c+1
```

```
    return c
```



Рисунок 2. Исходный код calculations.py

Задание 3. Команда ruff format --check . ничего не меняет, а только показывает, что изменилось бы: на рисунках 3 и 3.1 Ruff вывел разницу для calculations.py и итог «1 file would be reformatted, 2 files already formatted», задача завершилась с кодом 1. После ruff format . выведено «1 file reformatted, 2 files left unchanged» (рисунок 3.2). На рисунке 3.3 отформатированный файл: отступы в calc исправлены на 4 пробела, вокруг =, +, /, >, >=

расставлены пробелы. Импорты, имена и логика остались прежними — форматтер меняет только внешний вид кода.

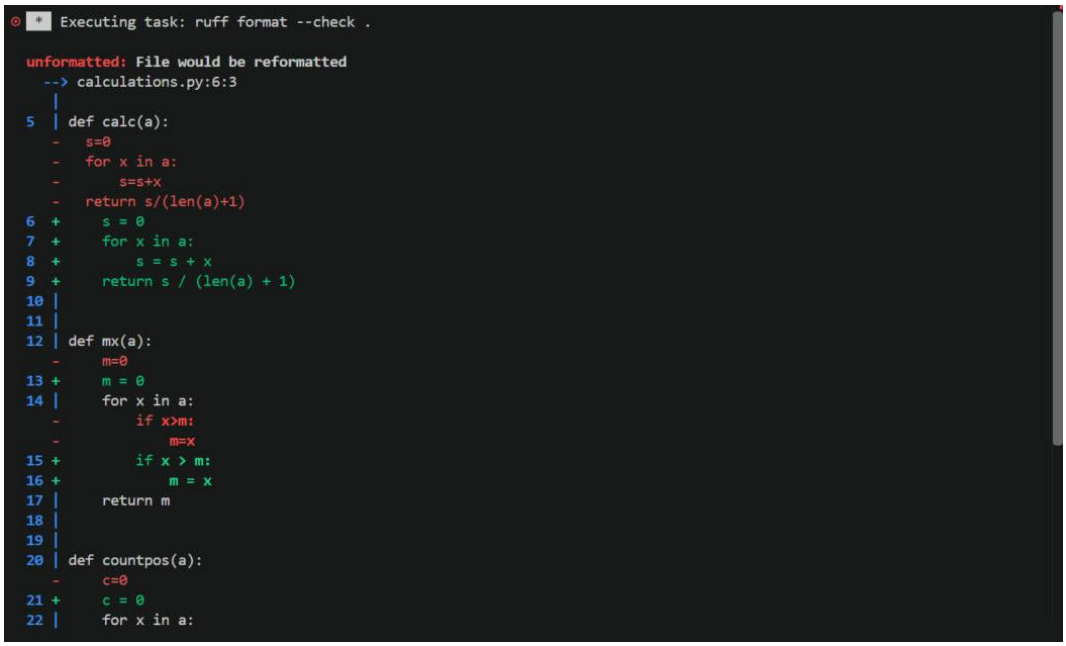


Рисунок 3. ruff format --check . (начало вывода)

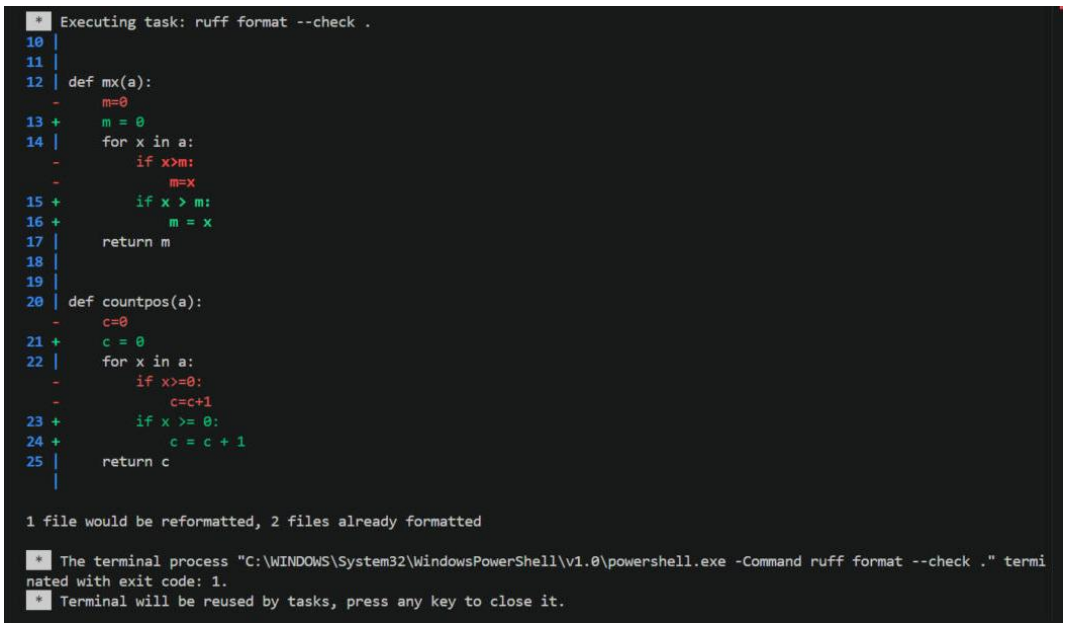


Рисунок 3.1. ruff format --check . (окончание вывода)

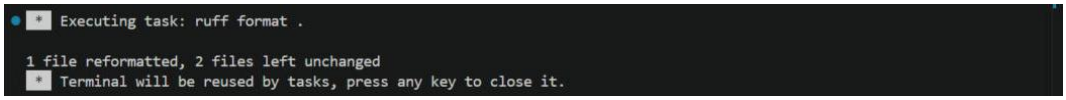


Рисунок 3.2. Результат ruff format .

```
import math
import json


def calc(a):
    s = 0
    for x in a:
        s = s + x
    return s / (len(a) + 1)


def mx(a):
    m = 0
    for x in a:
        if x > m:
            m = x
    return m


def countpos(a):
    c = 0
    for x in a:
        if x >= 0:
            c = c + 1
    return c
```



Рисунок 3.3. calculations.py после форматирования

Задание 4. ruff check . нашёл 4 проблемы (рисунки 4 и 4.1): I001 — блок импортов не отсортирован, F401 — math и json импортированы, но не используются, PLR1730 — конструкцию if x > m: m = x можно заменить на m = max(m, x). Все отмечены [*], то есть исправимы автоматически. ruff check . --fix вывел «Found 3 errors (3 fixed, 0 remaining)» (рисунок 4.2): число меньше

четырёх, потому что после удаления неиспользуемых импортов проблема сортировки исчезла сама. Повторный ruff check . — «All checks passed!» (рисунок 4.3). В итоговом файле импорты удалены, а в mx цикл переписан через max(); логические ошибки остались — линтер их не видит, поскольку код синтаксически корректен.

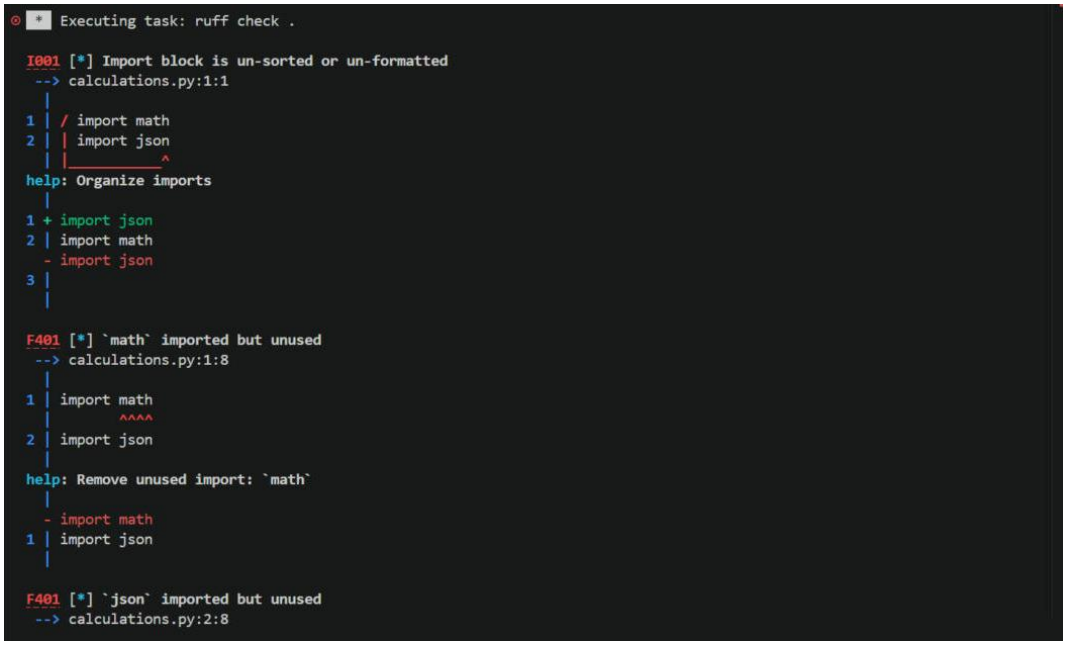


Рисунок 4. ruff check . — I001 и F401

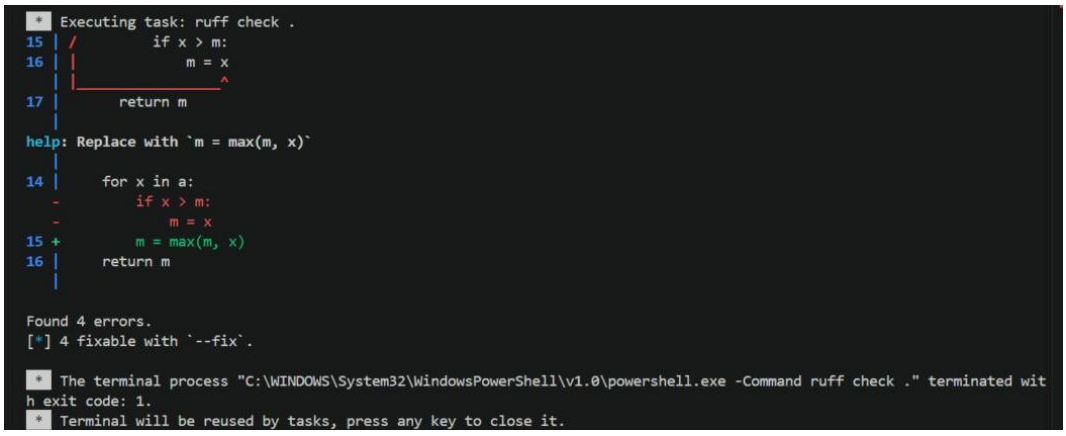


Рисунок 4.1. ruff check . — PLR1730 и итог

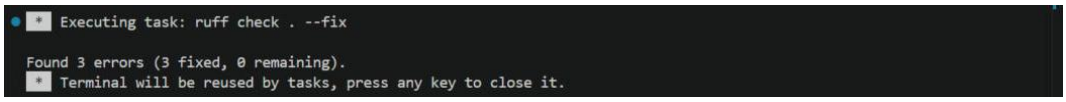


Рисунок 4.2. Автоматическое исправление ruff check . --fix

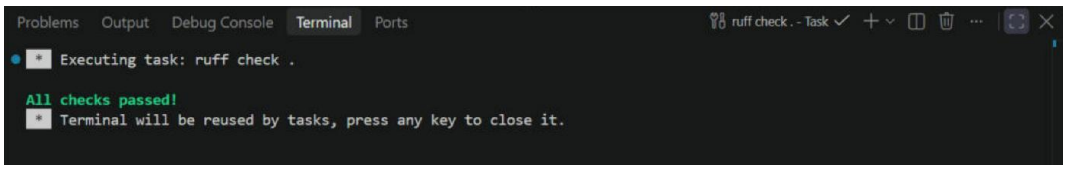


Рисунок 4.3. Повторная проверка без замечаний

```
def calc(a):
    s = 0
    for x in a:
        s = s + x
    return s / (len(a) + 1)


def mx(a):
    m = 0
    for x in a:
        m = max(m, x)
    return m


def countpos(a):
    c = 0
    for x in a:
        if x >= 0:
            c = c + 1
    return c
```

Задание 5. В main.py вызвана calc([10, 20, 30]); ожидалось 20, получено 15.0 (рисунок 5). Функция не исправлялась сразу: в calculations.py поставлена точка останова на строке return, программа запущена командой «Python Debugger: Debug Python File». На рисунке 5.1 выполнение остановлено на строке 7; на панели Variables и прямо в строке видно s = 60 (сумма элементов) и a = [10, 20, 30], то есть len(a) = 3, а выражение в делителе len(a) + 1 = 4. После Step Over на рисунке 5.2 отладчик показал возвращаемое значение (return) calc = 15.0 = 60 / 4. Причина ошибки — лишняя единица в

делителе: среднее равно сумме, делённой на количество элементов. После исправления на s / len(a) программа выводит 20.0 (рисунок 5.3).

```
from calculations import calc

result = calc([10, 20, 30])
print("Ожидаемое среднее значение: 20")
print("Фактическое среднее значение:", result)
```

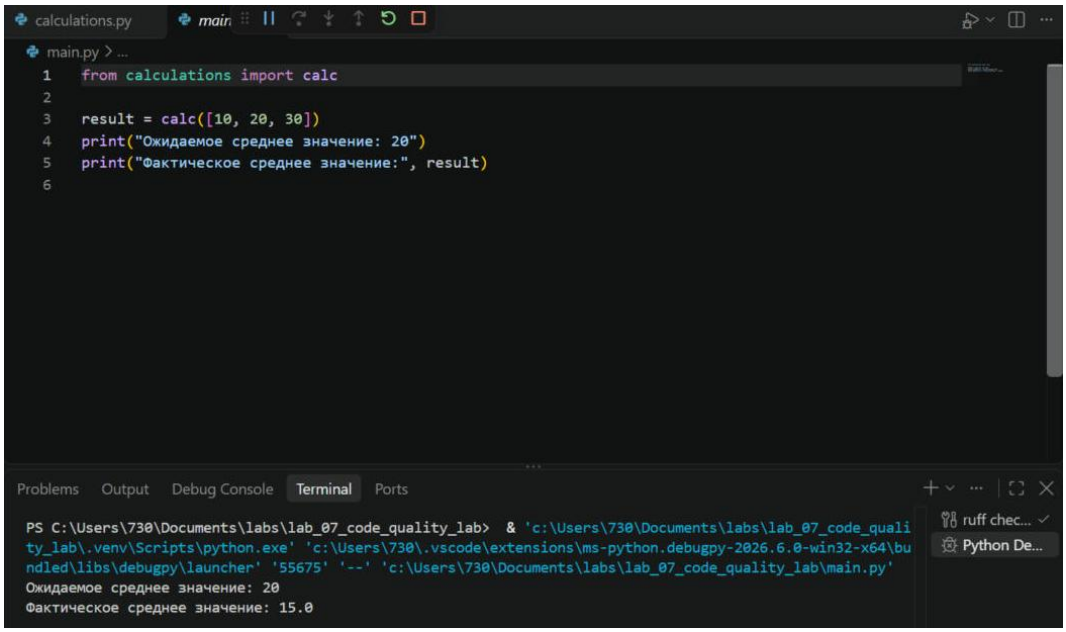


Рисунок 5. Неверное среднее значение

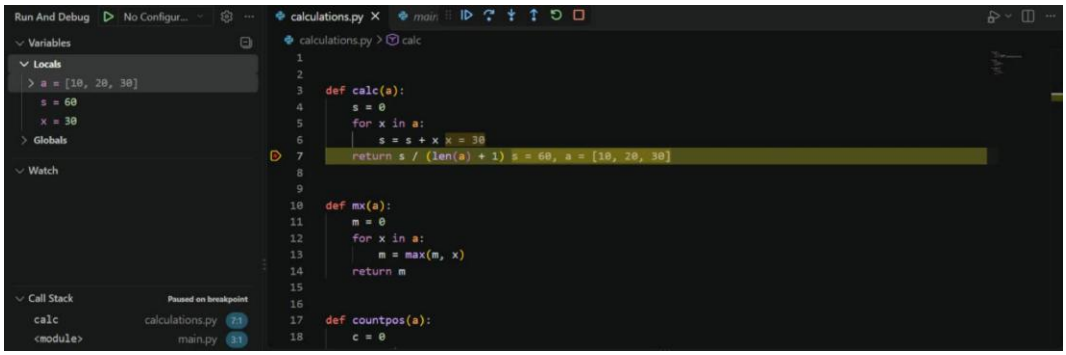


Рисунок 5.1. Остановка на точке останова в calc

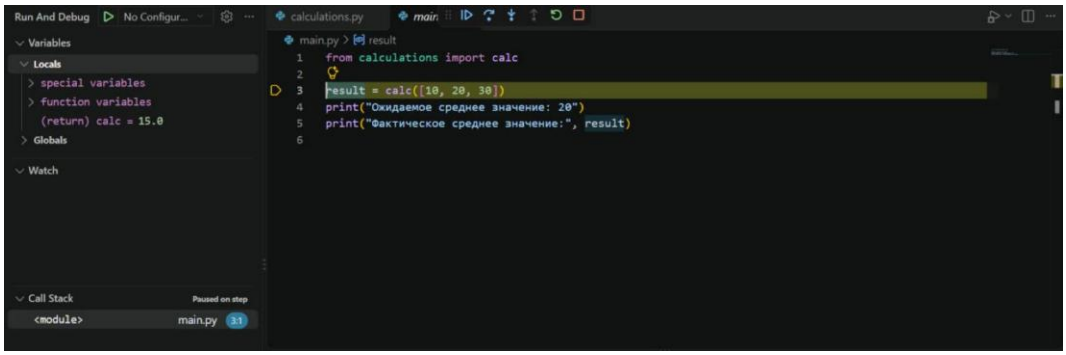


Рисунок 5.2. Возвращаемое значение после Step Over

```
def calc(a):
    s = 0
    for x in a:
        s = s + x
    return s / len(a)
```

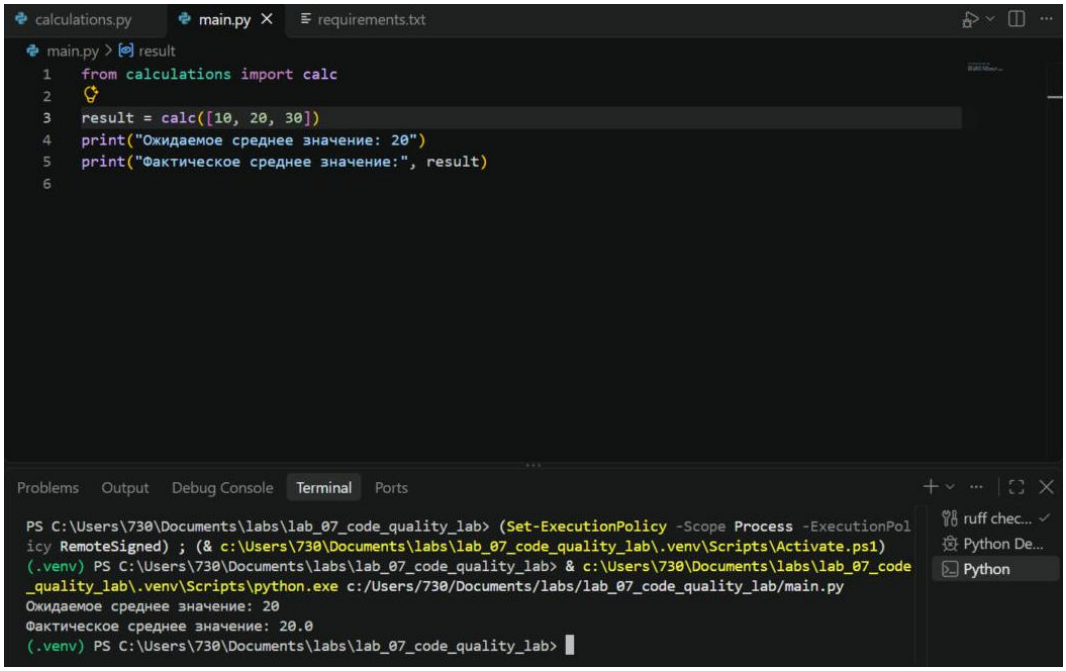


Рисунок 5.3. Среднее после исправления

Задание 6. Для mx([-10, -5, -20]) ожидается −5. Точка останова поставлена на строку m = max(m, x) внутри цикла, и выполнение продолжалось кнопкой Continue. На рисунках 6, 6.1 и 6.2 на трёх итерациях x равно −10, −5 и −20, а текущий максимум m всё время остаётся 0: начальное значение 0 больше любого элемента списка, и max(0, x) его не меняет. Программа вывела «Фактический максимум: 0» (рисунок 6.3). Алгоритм исправлен — начальным значением стал первый элемент: m = a[0]. После исправления результат −5 (рисунок 6.4).

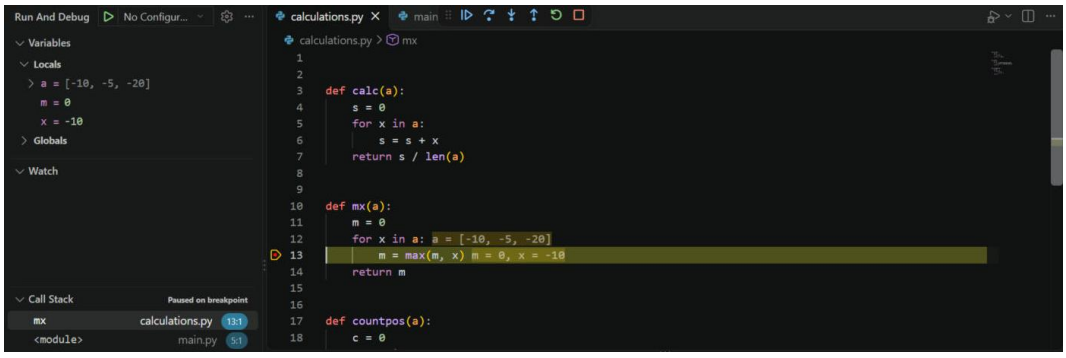


Рисунок 6. Первая итерация: x = −10, m = 0

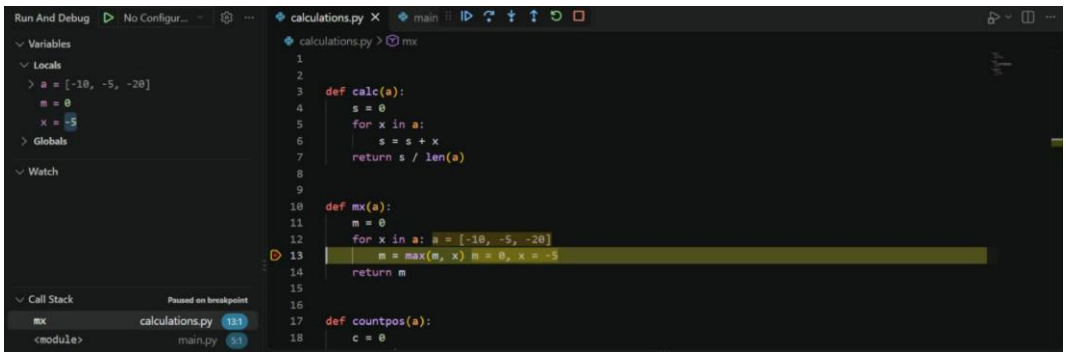


Рисунок 6.1. Вторая итерация: x = −5, m = 0

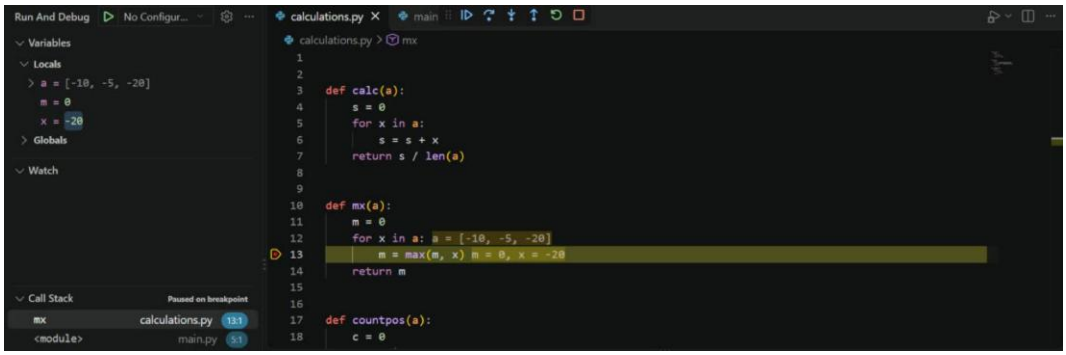


Рисунок 6.2. Третья итерация: x = −20, m = 0

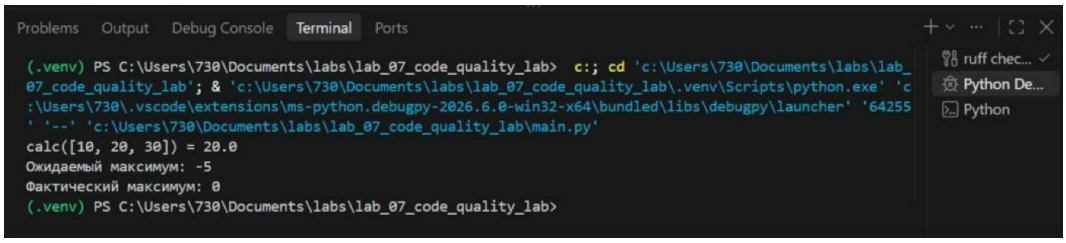


Рисунок 6.3. Неверный максимум

```
def mx(a):
    m = a[0]
    for x in a:
        m = max(m, x)
    return m
```

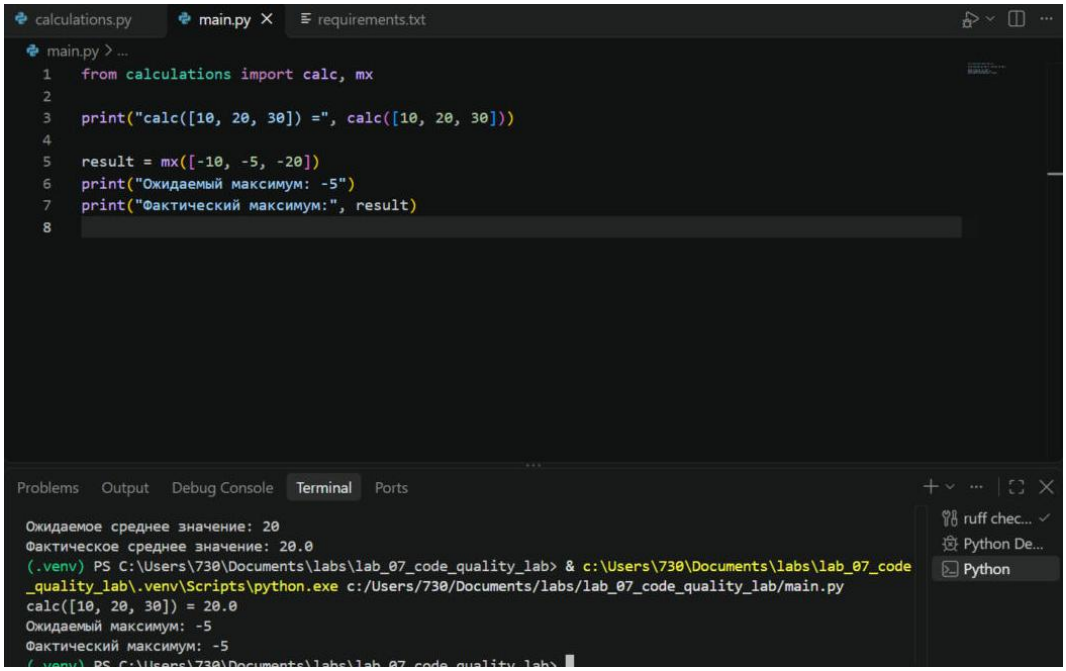


Рисунок 6.4. Максимум после исправления

Задание 7. countpos([-1, 0, 1]) вернула 2 вместо ожидаемой 1 (рисунок 7). Положительное число по определению строго больше нуля, а условие x >= 0

засчитывало и ноль. Условие заменено на x > 0, после чего результат равен 1 (рисунок 7.1).

```
from calculations import calc, countpos, mx

print("calc([10, 20, 30]) =", calc([10, 20, 30]))
print("mx([-10, -5, -20]) =", mx([-10, -5, -20]))

result = countpos([-1, 0, 1])
print("Ожидаемое количество положительных: 1")
print("Фактическое количество положительных:", result)
```

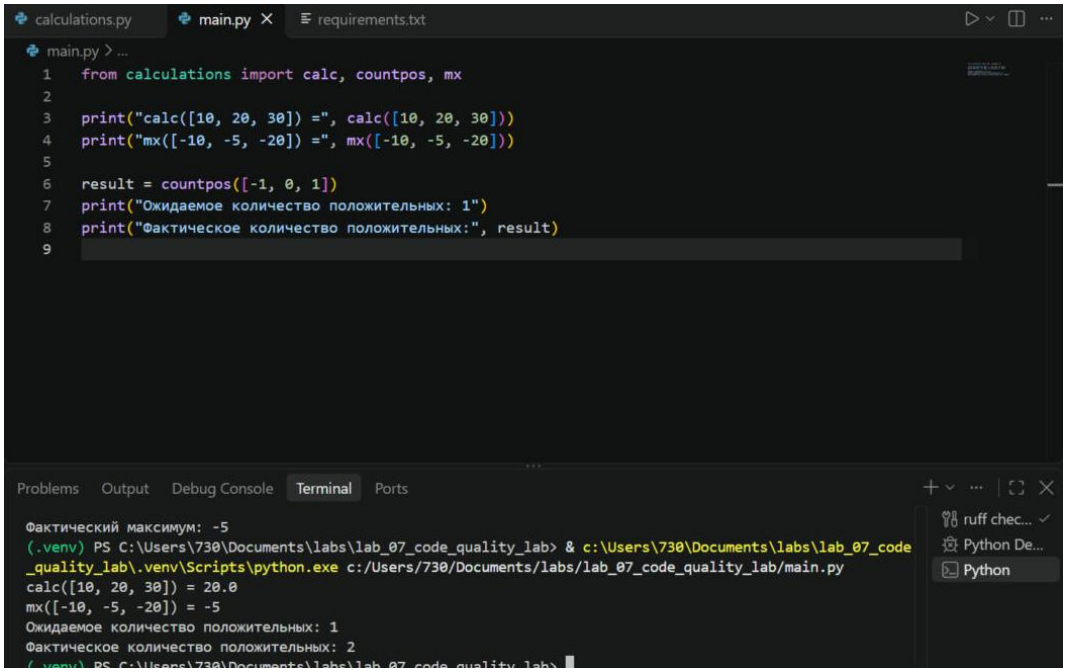


Рисунок 7. Неверный подсчёт положительных чисел



Рисунок 7.1. Подсчёт после исправления условия

Задание 8. Функции и переменные переименованы: calc → calculate_average, mx → find_max, countpos → count_positive, параметр a → numbers, s → total, m → maximum, c → count, x → number. На рисунке 8 показан новый

calculations.py и вызовы из main.py: результаты прежние — 20.0, −5 и 1, переименование поведение не изменило.

```
def calculate_average(numbers):
    total = 0
    for number in numbers:
        total = total + number
    return total / len(numbers)


def find_max(numbers):
    maximum = numbers[0]
    for number in numbers:
        maximum = max(maximum, number)
    return maximum


def count_positive(numbers):
    count = 0
    for number in numbers:
        if number > 0:
            count = count + 1
    return count
```

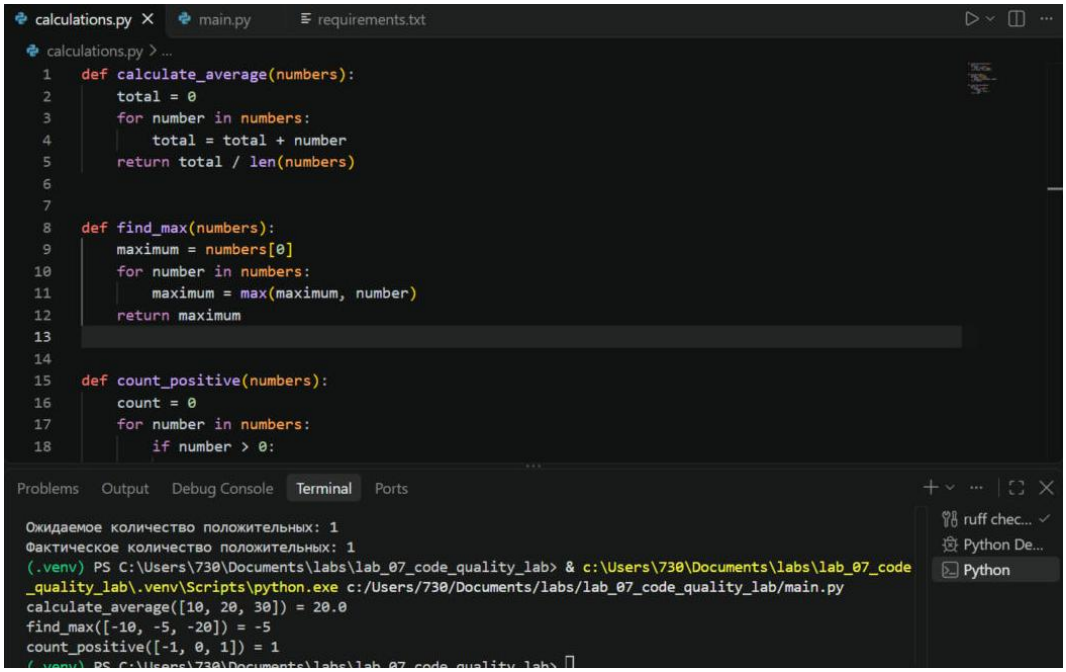


Рисунок 8. Функции после переименования

Задание 9. В main.py добавлены три проверки assert. На рисунке 9 все условия истинны, и программа вывела «Все проверки assert пройдены». Затем ожидаемое значение для find_max временно заменено на −10: на рисунке 9.1 возникло AssertionError с указанием строки 4 и проверяемого

выражения, а print не выполнился. После этого правильная проверка == -5 возвращена.

```
from calculations import calculate_average, count_positive,
find_max

assert calculate_average([10, 20, 30]) == 20
assert find_max([-10, -5, -20]) == -5
assert count_positive([-1, 0, 1]) == 1

print("Все проверки assert пройдены")
```

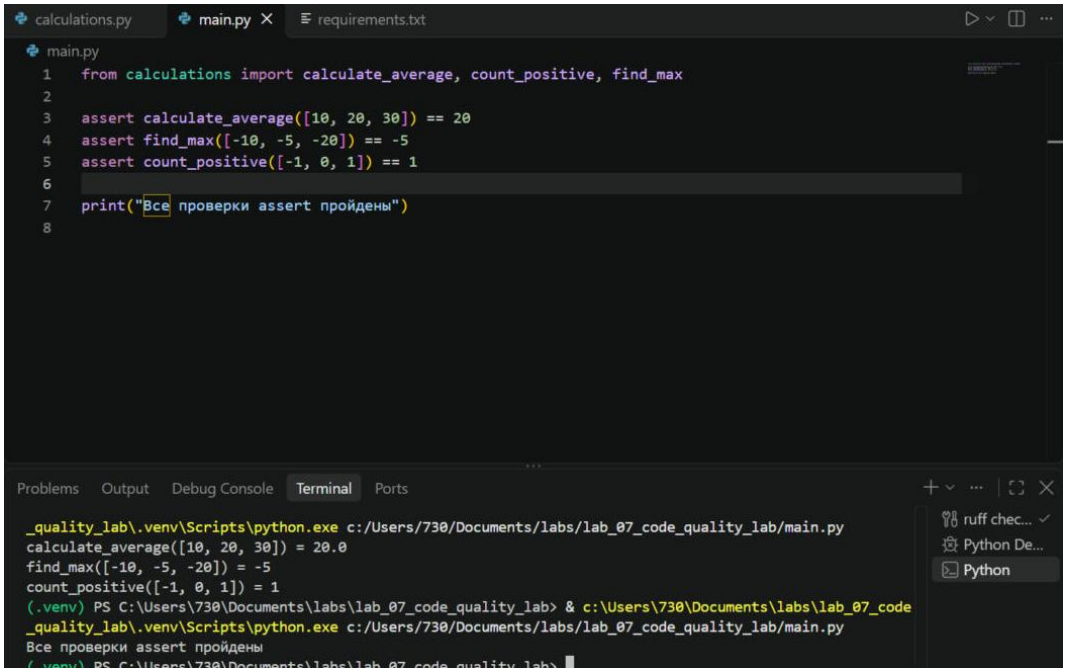


Рисунок 9. Проверки assert пройдены

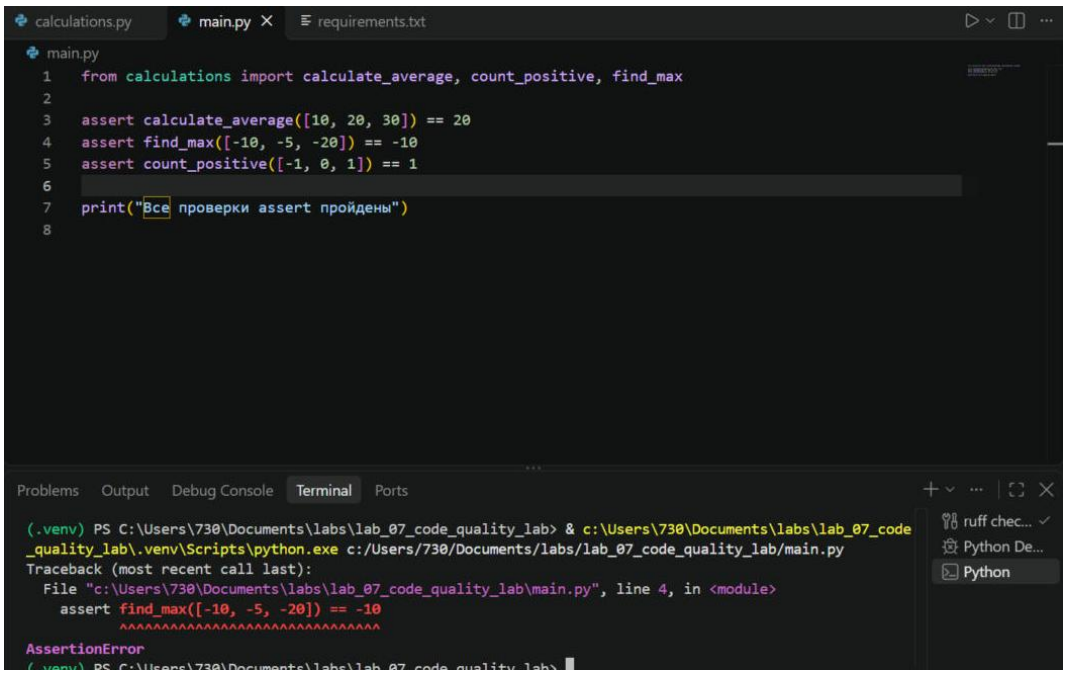


Рисунок 9.1. AssertionError при неверном ожидании

Задание 10. В tests/test_calculations.py созданы отдельные тестовые функции: для calculate_average — три набора данных, для find_max — два, для

count_positive — три. Команда pytest -v нашла 8 тестов, все прошли: «8 passed» (рисунок 10).

```
from calculations import calculate_average, count_positive,
find_max


def test_average_positive():
    assert calculate_average([10, 20, 30]) == 20


def test_average_single_value():
    assert calculate_average([5]) == 5


def test_average_negative():
    assert calculate_average([-10, 0, 10]) == 0


def test_find_max_negative():
    assert find_max([-10, -5, -20]) == -5


def test_find_max_mixed():
    assert find_max([3, 7, -1, 7]) == 7


def test_count_positive_with_zero():
    assert count_positive([-1, 0, 1]) == 1


def test_count_positive_all_negative():
    assert count_positive([-3, -2, -1]) == 0


def test_count_positive_all_positive():
    assert count_positive([1, 2, 3]) == 3
```

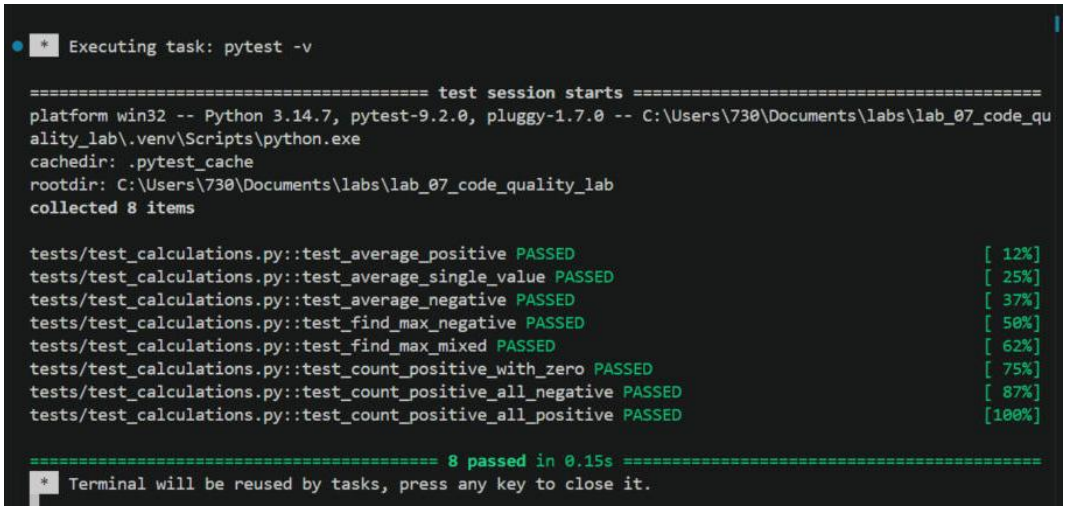


Рисунок 10. Результат pytest

Задание 11. В calculate_average временно возвращена ошибочная формула total / (len(numbers) + 1). pytest показал «2 failed, 6 passed» (рисунок 11): test_average_positive ожидал 20, а получил 15.0, test_average_single_value ожидал 5, а получил 2.5. Тест test_average_negative прошёл, потому что сумма [-10, 0, 10] равна 0 и при любом делителе результат 0 — это показывает, почему нужны несколько наборов данных. После возврата правильной формулы снова «8 passed» (рисунок 11.1).

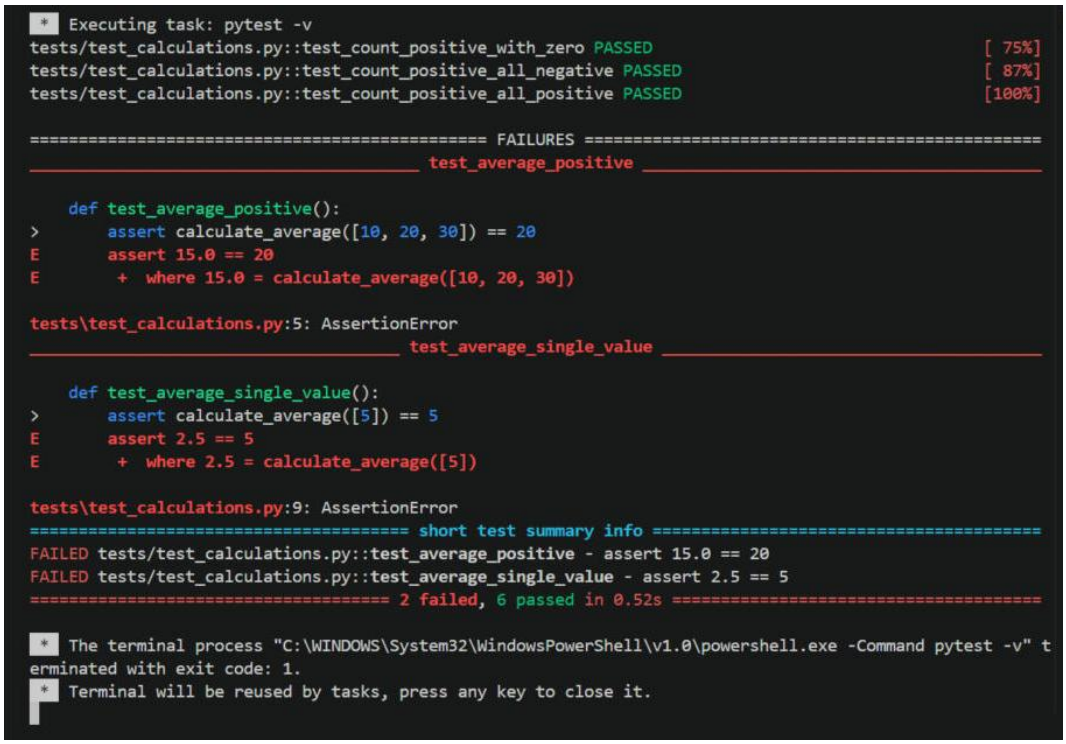


Рисунок 11. pytest обнаружил регрессию

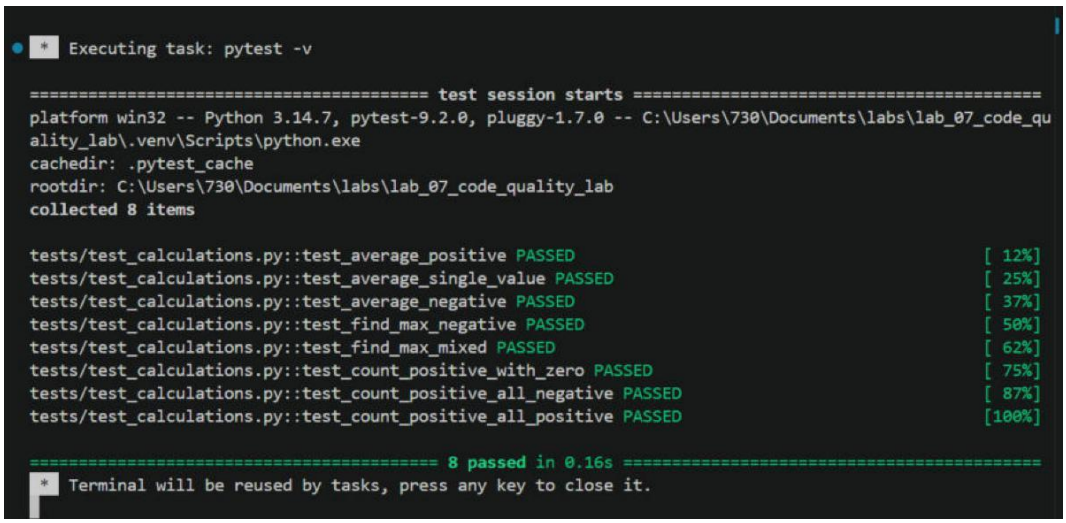


Рисунок 11.1. Все тесты проходят после исправления

Задание 12. calculate_average при пустом списке теперь выбрасывает ValueError("Список чисел пуст") вместо деления на ноль. Добавлен тест test_average_empty_list с pytest.raises(ValueError). На рисунке 12 — «9 passed», новый тест прошёл.

```
def calculate_average(numbers):
    if len(numbers) == 0:
        raise ValueError("Список чисел пуст")
    total = 0
    for number in numbers:
        total = total + number
    return total / len(numbers)


def test_average_empty_list():
    with pytest.raises(ValueError):
        calculate_average([])
```

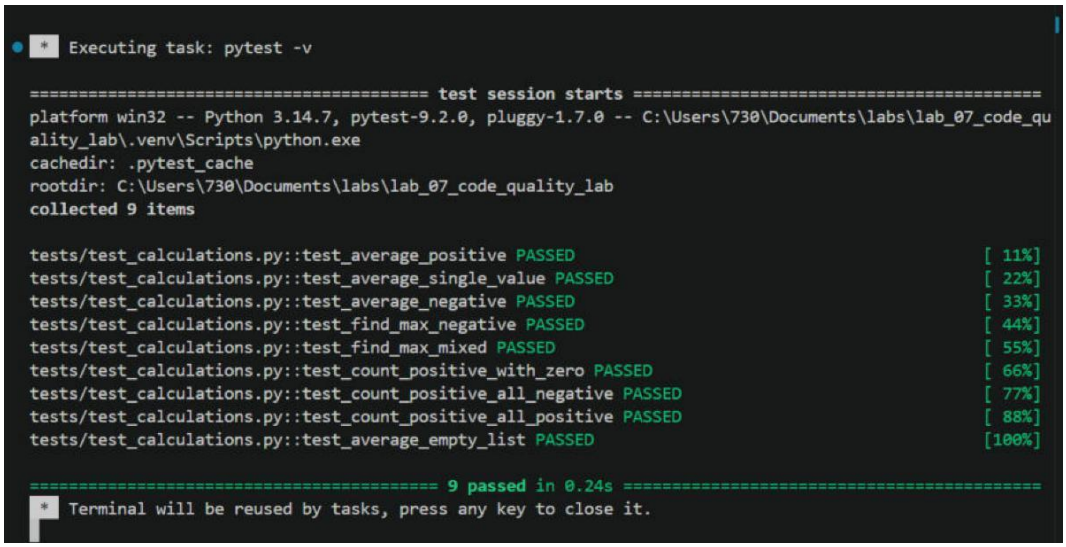


Рисунок 12. Тест на пустой список

Задание 13. Финальный рефакторинг: проверка пустого списка вынесена в функцию check_not_empty(), которую используют calculate_average и find_max (раньше find_max на пустом списке падал с IndexError); среднее считается встроенной sum(); find_max проходит по numbers[1:], не сравнивая первый элемент сам с собой; count_positive записан одним выражением sum(1 for ...). Демонстрация в main.py оформлена в функцию main() с проверкой if __name__ == "__main__". Добавлен тест test_find_max_empty_list. Последовательный запуск ruff format ., ruff check . и pytest (рисунок 13): «3

files left unchanged», «All checks passed!», «10 passed». На рисунке 13.1

итоговая программа выводит 20.0, −5 и 1.

```
def check_not_empty(numbers):
    if not numbers:
        raise ValueError("Список чисел пуст")


def calculate_average(numbers):
    check_not_empty(numbers)
    return sum(numbers) / len(numbers)


def find_max(numbers):
    check_not_empty(numbers)
    maximum = numbers[0]
    for number in numbers[1:]:
        maximum = max(maximum, number)
    return maximum


def count_positive(numbers):
    return sum(1 for number in numbers if number > 0)

from calculations import calculate_average, count_positive,
find_max


def main():
    measurements = [10, 20, 30]
    temperatures = [-10, -5, -20]
    values = [-1, 0, 1]

    print("Среднее значение:", calculate_average(measurements))
    print("Максимальная температура:", find_max(temperatures))
    print("Положительных чисел:", count_positive(values))


if __name__ == "__main__":
    main()
```

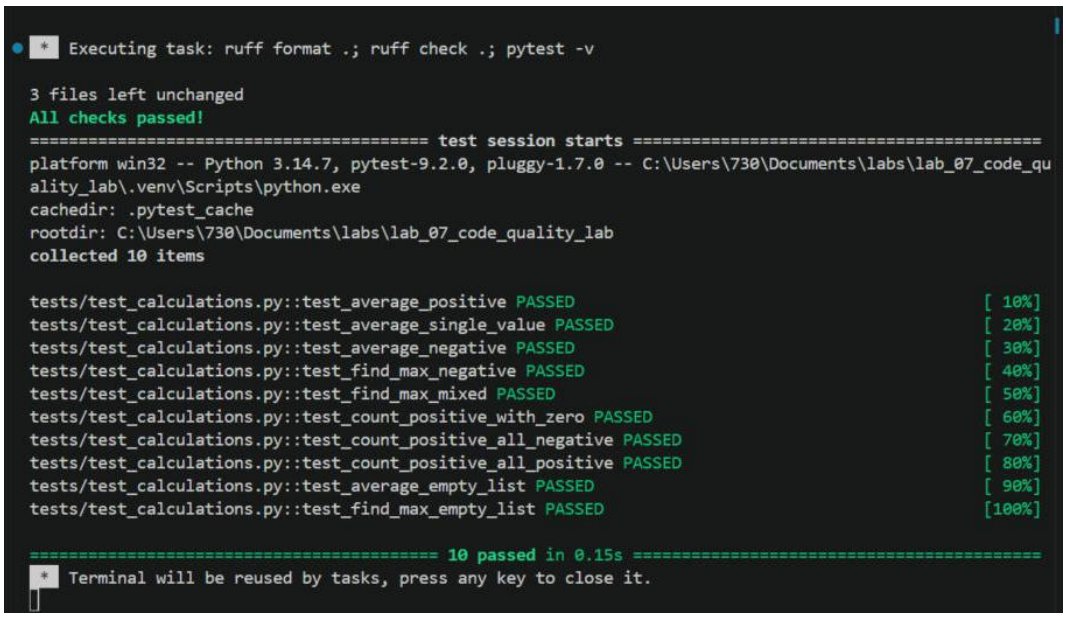


Рисунок 13. ruff format, ruff check и pytest после рефакторинга

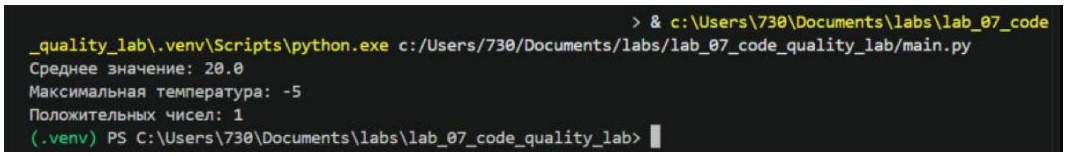


Рисунок 13.1. Работа итоговой программы

Задание 14. В calculations.py добавлена функция analize в исходном виде. Анализ показал проблемы: опечатка в имени (analize), однобуквенные имена и имена mx/mn, нет пробелов вокруг операторов, пустой список возвращает None вместо понятной ошибки. Главные логические ошибки — начальные mx = 0 и mn = 0: для списка только из отрицательных чисел максимум получается 0, а для списка только из положительных — минимум 0. На

рисунке 14 это видно: analize([3, 1, 2]) = (0, 3, 2.0), analize([-5, -1, -3]) = (-5, 0, -3.0), analize([10, 20]) = (0, 20, 15.0), analize([]) = None.

```
def analize(a):
    if len(a)==0:
        return None
    s=0
    mx=0
    mn=0
    for x in a:
        s+=x
        if x>mx:
            mx=x
        if x<mn:
            mn=x
    return mn,mx,s/len(a)
```



Рисунок 14. Результаты исходной функции analize

Ruff отформатировал файл («1 file reformatted») и нашёл две проблемы PLR1730 в цикле analize (рисунок 14.1); тесты на этом этапе ещё старые — 10 passed (рисунок 14.2). Затем функция переписана как analyze_numbers(numbers): проверка пустого списка через check_not_empty() (ValueError), минимум и максимум начинаются с первого элемента, обновляются через min() и max(), среднее берётся из calculate_average(). Написано шесть тестов: смешанные числа, только отрицательные, только положительные, одно значение, список с нулём и пустой список. Итоговая проверка на рисунке 14.3: «3 files left unchanged», «All checks passed!», «16

passed». Программа на рисунке 14.4 выводит верные минимум, максимум и среднее для всех трёх наборов.

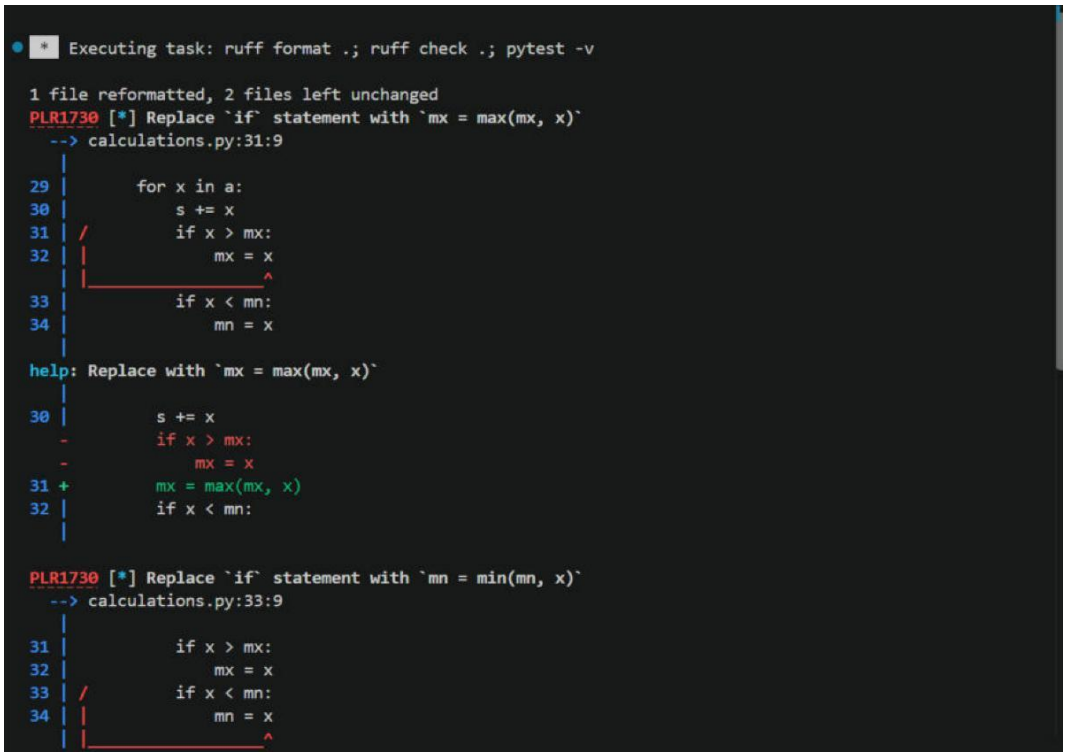


Рисунок 14.1. Замечания Ruff к функции analize

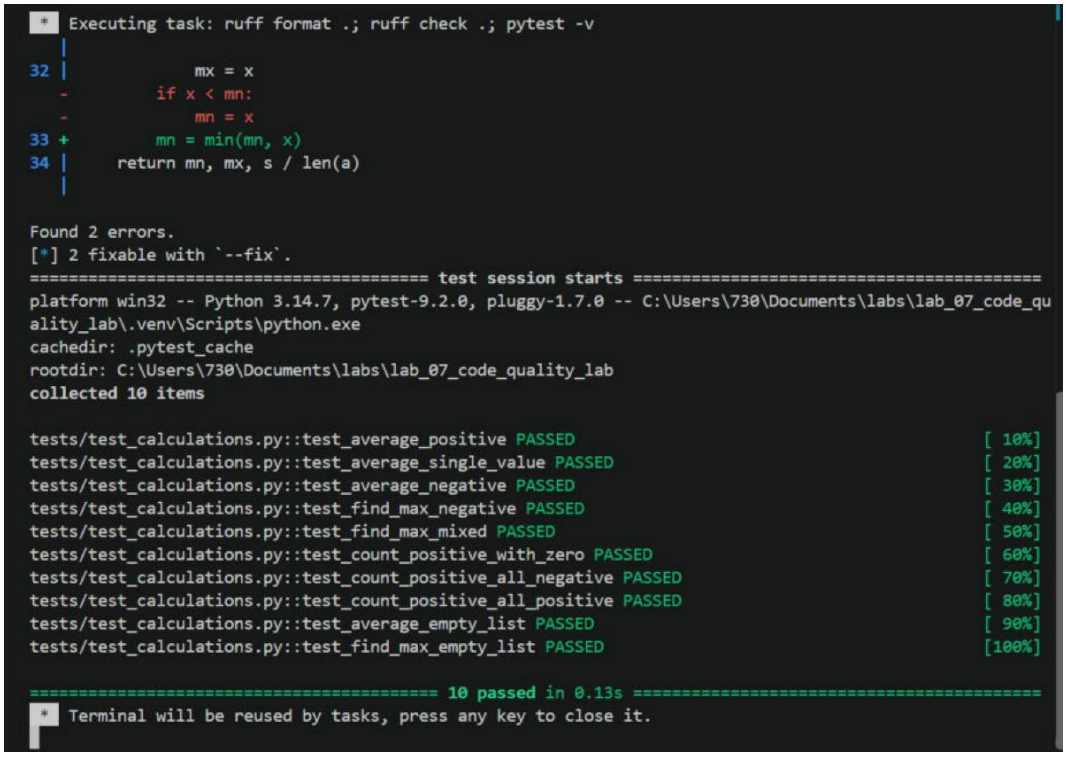


Рисунок 14.2. Тесты до добавления новых проверок

```
def analyze_numbers(numbers):
    """Возвращает минимум, максимум и среднее значение
списка."""
    check_not_empty(numbers)
    minimum = maximum = numbers[0]
    for number in numbers[1:]:
        minimum = min(minimum, number)
        maximum = max(maximum, number)
    return minimum, maximum, calculate_average(numbers)

def test_analyze_mixed():
    assert analyze_numbers([3, 1, 2]) == (1, 3, 2)


def test_analyze_all_negative():
    assert analyze_numbers([-5, -1, -3]) == (-5, -1, -3)


def test_analyze_all_positive():
    assert analyze_numbers([10, 20]) == (10, 20, 15)


def test_analyze_single_value():
    assert analyze_numbers([7]) == (7, 7, 7)


def test_analyze_with_zero():
    assert analyze_numbers([-2, 0, 2]) == (-2, 2, 0)


def test_analyze_empty_list():
    with pytest.raises(ValueError):
        analyze_numbers([])
```

```
* Executing task: ruff format .; ruff check .; pytest -v

3 files left unchanged
All checks passed!
========================================= test session starts =========================================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\730\Documents\labs\lab_07_code_qu
ality_lab\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\730\Documents\labs\lab_07_code_quality_lab
collected 16 items

tests/test_calculations.py::test_average_positive PASSED                                    [  6%]
tests/test_calculations.py::test_average_single_value PASSED                                [ 12%]
tests/test_calculations.py::test_average_negative PASSED                                    [ 18%]
tests/test_calculations.py::test_find_max_negative PASSED                                   [ 25%]
tests/test_calculations.py::test_find_max_mixed PASSED                                      [ 31%]
tests/test_calculations.py::test_count_positive_with_zero PASSED                            [ 37%]
tests/test_calculations.py::test_count_positive_all_negative PASSED                         [ 43%]
tests/test_calculations.py::test_count_positive_all_positive PASSED                         [ 50%]
tests/test_calculations.py::test_average_empty_list PASSED                                  [ 56%]
tests/test_calculations.py::test_find_max_empty_list PASSED                                 [ 62%]
tests/test_calculations.py::test_analyze_mixed PASSED                                       [ 68%]
tests/test_calculations.py::test_analyze_all_negative PASSED                                [ 75%]
tests/test_calculations.py::test_analyze_all_positive PASSED                                [ 81%]
tests/test_calculations.py::test_analyze_single_value PASSED                                [ 87%]
tests/test_calculations.py::test_analyze_with_zero PASSED                                   [ 93%]
tests/test_calculations.py::test_analyze_empty_list PASSED                                  [100%]

========================================= 16 passed in 0.15s ==========================================
* Terminal will be reused by tasks, press any key to close it.
```

Рисунок 14.3. Итоговые ruff format, ruff check и pytest

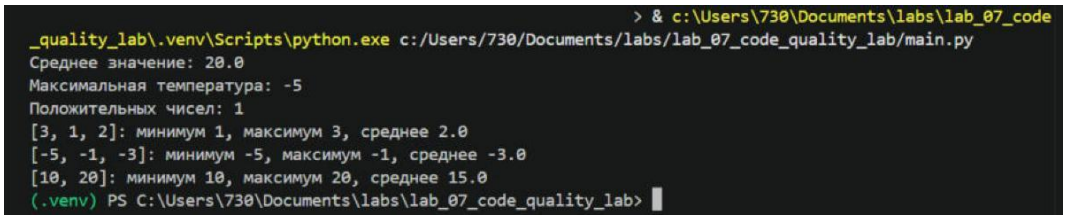


Рисунок 14.4. Работа функции analyze_numbers

Вывод

В ходе лабораторной работы изучены принципы качественного Python-кода. Исходный модуль содержал неиспользуемые импорты, неправильные отступы, отсутствие пробелов вокруг операторов, непонятные имена и три логические ошибки; все проблемы были найдены и исправлены, а функции получили говорящие имена calculate_average, find_max и count_positive в соответствии с PEP 8.

Использован Ruff: ruff format привёл код к единому стилю, не меняя логику, а ruff check нашёл неиспользуемые импорты, неотсортированные импорты и упрощаемые конструкции и исправил их автоматически. При этом статический анализ показал «All checks passed!» для кода, в котором ещё оставались логические ошибки, — линтер не проверяет правильность алгоритма.

Логические ошибки найдены отладчиком VS Code: точки останова, Continue, Step Over и панель Variables показали, что среднее делится на len(a) + 1, а максимум не меняется из-за начального значения 0. Проверки assert и тесты pytest закрепили правильное поведение; при временном возврате ошибки pytest сразу указал упавшие тесты с ожидаемыми и фактическими значениями, то есть тесты защищают от регрессии.

Выполнен рефакторинг: общая проверка пустого списка вынесена в check_not_empty(), используются встроенные sum(), min() и max(), сценарий оформлен в main(). Самостоятельно разобрана и переписана функция analize: исправлены начальные значения минимума и максимума и обработка пустого

списка, написано шесть тестов. Итоговый проект проходит ruff format, ruff check и все 16 тестов pytest.